

# QA-ISystem Architecture Reference

---

## QA-ISystem: Complete Architecture & Design Reference

---

**Audience:** *This document is written for someone who is new to coding and solution design. Every concept is explained from first principles before diving into how it is applied here.*

---

### 1. What Problem Does This System Solve?

---

When a developer writes new code and opens a Pull Request (PR), someone (or something) needs to answer:

*“Does the test suite need updating? And if so, what tests should be written?”*

Traditionally, a human QA engineer looks at the changed code, figures out which components were affected, checks whether they are covered by tests, writes BDD scenarios, generates test code, runs it, fixes it, and opens a PR. This process can take hours to days.

**QA-ISystem automates this entire workflow:**

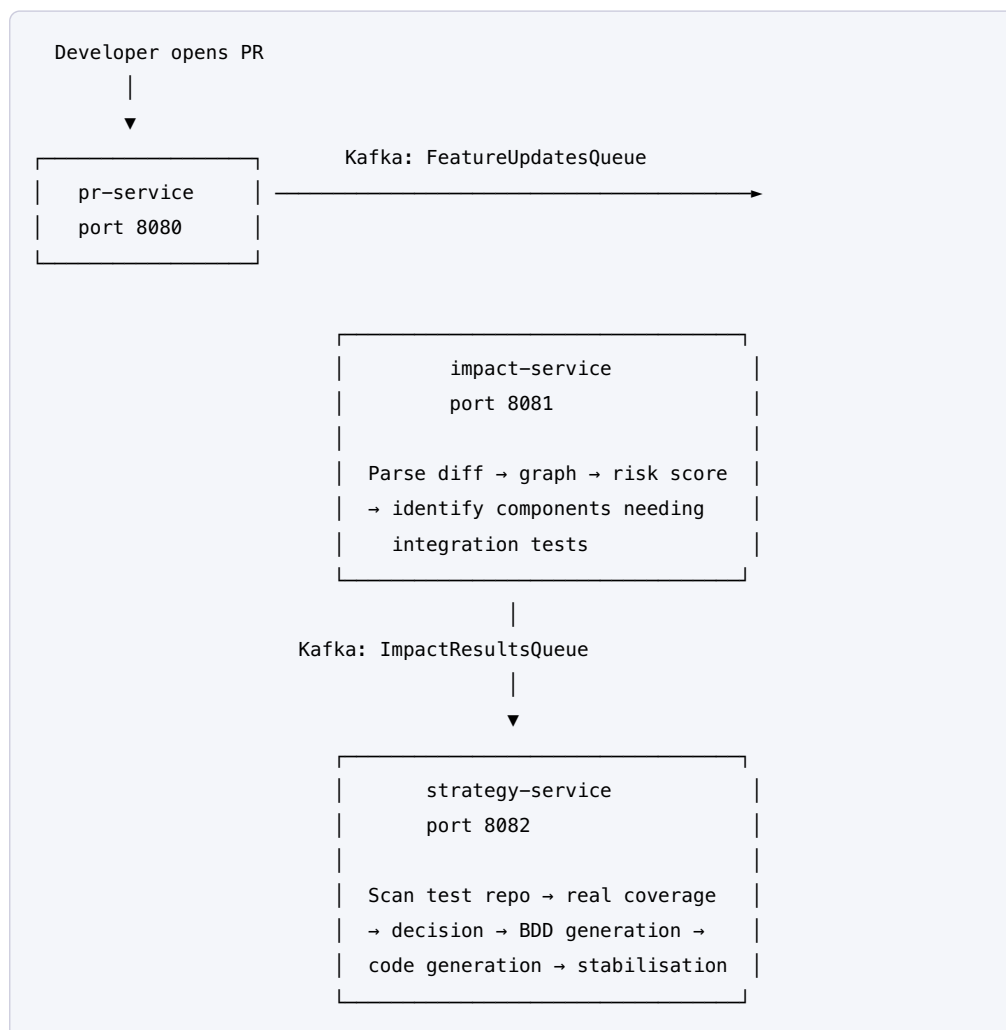
1. A PR is submitted → the system receives it
2. The system analyses the code change (which files changed, what kind of change, how risky it is)

3. It checks whether the changed components already have integration or end-to-end tests
  4. It makes a decision: skip / update existing tests / create new tests
  5. It generates BDD (Gherkin) scenarios for human review
  6. After human approval, it generates executable test code
  7. It runs the test, self-heals minor failures, and opens a final PR
- 

## 2. System Overview

---

The system is split into **three microservices** and one **shared library** ( common ). Each service runs independently and communicates via **Apache Kafka** (a message queue).



### 3. What Is Apache Kafka and Why Is It Used Here?

---

**Apache Kafka** is a distributed message queue. Think of it like a post box: one service puts a message in, another service picks it up — but they don't need to be running at the same time or at the same speed.

#### Why Kafka instead of direct HTTP calls?

| Option                                         | Problem                                                                                                                                                                                                                                       |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pr-service calls impact-service directly       | If impact-service is slow or down, pr-service hangs and the developer's webhook request times out                                                                                                                                             |
| Impact-service calls strategy-service directly | Same coupling: one slow step blocks everything                                                                                                                                                                                                |
| <b>Kafka between each step</b>                 | Each service processes at its own pace. A slow strategy-service does not block a fast impact-service. Each service can have multiple consumers in parallel. Messages are durable — if strategy-service crashes, it picks up where it left off |

---

#### Key Kafka concepts used here

| Term                  | Meaning in this system                                                                                                                                                             |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Topic</b>          | A named queue. <code>FeatureUpdatesQueue</code> , <code>ImpactResultsQueue</code> , <code>TestScriptsQueue</code>                                                                  |
| <b>Producer</b>       | A service that puts messages in a topic                                                                                                                                            |
| <b>Consumer</b>       | A service that reads messages from a topic                                                                                                                                         |
| <b>Consumer group</b> | Multiple instances of the same service that share the work (e.g. <code>impact-service-group</code> with 3 threads)                                                                 |
| <b>Partition key</b>  | Messages with the same key (e.g. <code>prId</code> ) go to the same partition and are processed in order                                                                           |
| <b>Manual ack</b>     | The consumer explicitly tells Kafka "I have processed this message" ( <code>AckMode.MANUAL_IMMEDIATE</code> ). If the service crashes before acking, Kafka re-delivers the message |

---

## 4. The `common` Module

---

The `common` module is a shared Java library (JAR) that is compiled once and used as a dependency by all three services. It contains all the **data models** — plain Java objects that carry information between services.

### Why share models?

If each service had its own definition of `PullRequest`, they might disagree on field names or types. A shared `common` module means they all speak the same language.

### Key models

#### `PullRequest`

The input to the system. Represents a code change that a developer wants reviewed.

```
PullRequest
├─ prId          "PR-A1B2C3D4"
├─ title         "feat: Add payment gateway"
├─ author        "dev@example.com"
├─ repositoryName "payment-service"
├─ sourceBranch  "feature/payments"
├─ targetBranch  "main"
├─ rawDiffContent "diff --git a/src/PaymentService.java ..."
└─ diffs         List<GitDiff> (pre-parsed, optional)
```

#### `GitDiff`

Represents one changed file. A PR touching 5 files produces 5 `GitDiff` objects.

```
GitDiff
├─ filePath      "src/main/java/PaymentService.java"
├─ diffType      ADDED | MODIFIED | DELETED | RENAMED
├─ linesAdded    45
├─ linesDeleted  12
├─ isTestFile    false
└─ hunks         List<DiffHunk>
    └─ lines: List<DiffLine>
```

```
└─ type: ADDED | REMOVED | CONTEXT
└─ content: the actual line of code
```

### ImpactEnvelope

The output of `impact-service`. Contains everything the strategy layer needs.

```
ImpactEnvelope
└─ envelopeId          (UUID)
└─ prId
└─ impactedComponents  List<ImpactedComponent>
    └─ componentName, filePath, type (CONTROLLER/SERVICE/...), impactScore,
        callers, callees
└─ detectedChangeTypes List<ChangeType> (API_CHANGE, BUG_FIX,
    SECURITY_FIX, ...)
└─ overallRiskScore    0.0 - 1.0
└─ riskLevel           LOW | MEDIUM | HIGH | CRITICAL
└─ coverageReport      CoverageReport (level=UNKNOWN at this point)
└─ suggestedTestAreas  List<String> (component names needing coverage)
└─ changesSummary      Human-readable one-line description
```

### CoverageReport

The two-phase coverage model — starts UNKNOWN in `impact-service`, gets resolved to real data in `strategy-service`.

```
CoverageReport
└─ source              UNKNOWN | REPO_SCAN
└─ level              GOOD | PARTIAL | NONE | UNKNOWN
└─ coverageRatio       0.0 - 1.0 (meaningful only after REPO_SCAN)
└─ testedComponents   List of component names WITH existing integration tests
└─ untestedComponents List of component names WITHOUT integration tests
└─ requiredTestTypes  ["API", "INTEGRATION", "E2E"]
└─ existingTestFiles  List of test filenames found in the repo
└─ requiresNewTests   boolean
```

---

## 5. pr-service — Phase 0: Ingestion

---

**Port:** 8080 | **Role:** Receive PR events, validate, enrich, publish to Kafka

### What it does

1. Developer (or CI webhook) sends a POST request with PR details

2. Service validates required fields (title, author, repository name)
3. Service fills in defaults for optional fields (prId, targetBranch, timestamps)
4. Service publishes the enriched `PullRequest` as JSON to `FeatureUpdatesQueue`

## Design decisions

### Why a separate service for just “receive and forward”?

This service acts as a **gateway**. It: - Protects the rest of the system from invalid input - Decouples the external caller (webhook, CI) from the internal implementation - Can be scaled independently if PR volume is high

**No custom Kafka configuration needed here.** pr-service only produces messages (never consumes). Spring Boot auto-creates a `KafkaTemplate` from the YAML config, with no custom beans required. This is intentional simplicity — only services that *consume* Kafka messages need the manual-ack container setup.

## Classes

| Class                               | Role                                                                                                                 |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>PrServiceApplication</code>   | Spring Boot entry point — starts the application                                                                     |
| <code>PRController</code>           | HTTP layer: 4 endpoints ( <code>/webhook</code> , <code>/submit</code> , <code>/demo</code> , <code>/health</code> ) |
| <code>PRService</code>              | Business logic: enrich + validate + publish                                                                          |
| <code>FeatureUpdatesProducer</code> | Kafka layer: serialise + send to <code>FeatureUpdatesQueue</code>                                                    |

## Endpoint summary

| Endpoint                          | Use case                   | Notes                          |
|-----------------------------------|----------------------------|--------------------------------|
| <code>POST /api/pr/webhook</code> | GitHub webhook integration | Lenient — no strict validation |
| <code>POST /api/pr/submit</code>  | Programmatic submission    |                                |

| Endpoint                        | Use case            | Notes                                             |
|---------------------------------|---------------------|---------------------------------------------------|
|                                 |                     | Strict <code>@Valid</code> annotation enforcement |
| POST <code>/api/pr/demo</code>  | Quick local testing | Uses a built-in JWT auth sample PR                |
| GET <code>/api/pr/health</code> | Health check        | Returns <code>{"status":"UP"}</code>              |

## 6. impact-service — Phase 1: Deterministic Analysis

**Port:** 8081 | **Role:** Parse the diff, classify the change, score risk, identify coverage gaps

***No AI here.** Every step is a deterministic algorithm. The same diff always produces the same output.*

### The 5-step pipeline

```
PullRequest → [GitDiffParser] → [DependencyGraph] → [ChangeTypeDetector]
              → [RiskScorer]    → [TestCoverageService] → ImpactEnvelope
```

### Step 1: GitDiffParser — Reading the raw diff

A **git diff** is a plain-text format that shows what changed in a file.

Example:

```
diff --git a/src/PaymentService.java b/src/PaymentService.java
@@ -10,6 +10,8 @@ public class PaymentService {
    private final PaymentRepository repo;
+   private final AuditService audit;
+
    public void processPayment(Payment p) {
```

The parser reads this line by line: - `diff --git` → start of a new file -  
`@@ ... @@` → start of a hunk (a changed region within the file) - Line  
starting with `+` → added line - Line starting with `-` → removed line -  
Line starting with space → unchanged context line

**Output:** A structured `List<GitDiff>` where each diff has typed hunks  
and lines, plus metadata like `linesAdded`, `linesDeleted`, and  
whether it is a test file.

## Step 2: DependencyGraph — Who depends on what?

For every changed file, `DependencyGraph` extracts Java `import`  
statements from the changed lines and builds a map of:

```
ClassName → [classesItImports]
```

It then identifies **callers** — other changed classes that import this one.  
A class with many callers has a wider impact (“blast radius”) and gets a  
higher impact score.

**ComponentType detection** (from file path keywords):

| Path keyword                                                   | ComponentType | Significance                             |
|----------------------------------------------------------------|---------------|------------------------------------------|
| <code>controller</code>                                        | CONTROLLER    | HTTP endpoint — highest user-facing risk |
| <code>service</code>                                           | SERVICE       | Business logic                           |
| <code>repositor</code> or <code>dao</code>                     | REPOSITORY    | Database access                          |
| <code>model</code> , <code>entity</code> ,<br><code>dto</code> | MODEL         | Data structure                           |
| <code>config</code>                                            | CONFIG        | System configuration                     |
| <code>test</code> or <code>spec</code>                         | TEST          | Test file                                |
| <code>util</code> or <code>helper</code>                       | UTILITY       | Helper code                              |

**Impact score formula:**

```
base          = min(1.0, (linesAdded + linesDeleted) / 200)  
callerBonus   = min(0.3, numberOfCallers × 0.1)  
impactScore   = min(1.0, base + callerBonus)
```



### Step 3: ChangeTypeDetector — What kind of change is this?

Uses regex patterns on file paths and diff content to classify the change. Multiple types can be detected per PR.

#### File path patterns:

| File path matches                   | ChangeType           |
|-------------------------------------|----------------------|
| application.yml, .env, config/      | CONFIGURATION_CHANGE |
| pom.xml, build.gradle, package.json | DEPENDENCY_UPDATE    |
| migration, flyway, liquibase, .sql  | DATABASE_CHANGE      |

#### Content keyword patterns (applied to changed lines only):

| Keywords found                                   | ChangeType              |
|--------------------------------------------------|-------------------------|
| TODO, FIXME, bug, fix, patch                     | BUG_FIX                 |
| @Deprecated, rename, refactor                    | REFACTORING             |
| security, auth, token, password, secret          | SECURITY_FIX            |
| performance, cache, async, parallel              | PERFORMANCE_IMPROVEMENT |
| @RestController, @GetMapping, @PostMapping, etc. | API_CHANGE              |
| migration, ALTER TABLE, CREATE TABLE, flyway     | DATABASE_CHANGE         |

**Special rules:** - New file that is not a test → NEW\_FEATURE - Deleted file → REFACTORING - More than 50 lines deleted → BREAKING\_CHANGE - Nothing detected → default NEW\_FEATURE

### Step 4: RiskScorer — How risky is this change?

Produces a normalised score from **0.0** (safe) to **1.0** (maximum risk), using four weighted factors:

```
riskScore = (0.25 × churnScore)
            + (0.30 × changeTypeScore)
            + (0.25 × componentScore)
            + (0.20 × coverageScore)
```

#### Factor 1: Churn score (25%)

```
churnScore = min(1.0, (linesAdded + linesDeleted) / 300)
```

More lines changed = more things that could go wrong. 300 lines saturates the score.

**Factor 2: Change type severity (30%)** Average severity across detected change types:

| ChangeType                         | Severity |
|------------------------------------|----------|
| BREAKING_CHANGE                    | 1.0      |
| SECURITY_FIX                       | 0.9      |
| DATABASE_CHANGE                    | 0.8      |
| API_CHANGE                         | 0.7      |
| DEPENDENCY_UPDATE                  | 0.6      |
| NEW_FEATURE / BUG_FIX              | 0.5      |
| PERFORMANCE_IMPROVEMENT            | 0.4      |
| CONFIGURATION_CHANGE / REFACTORING | 0.3      |

**Factor 3: Component criticality (25%)** Average per component type, plus a bonus for wide blast radius:

| ComponentType | Weight |
|---------------|--------|
| CONTROLLER    | 0.8    |
| REPOSITORY    | 0.7    |
| SERVICE       | 0.6    |
| CONFIG        | 0.5    |
| MODEL         | 0.4    |
| TEST          | 0.1    |
| default       | 0.3    |

Caller bonus:  $\min(0.2, \text{avgCallers} \times 0.05)$  — a class called by many others is riskier to change.

**Factor 4: Coverage uncertainty (20%)**

```
integrationTestable = count of CONTROLLER + SERVICE + REPOSITORY components
nonTestComponents   = all components except TEST and UTILITY
coverageScore        = min(0.9, integrationTestable / nonTestComponents × 0.9)
```

The more integration-testable components changed, the higher the coverage uncertainty. Capped at 0.9 — never declares maximum risk without a real repo scan.

### Risk levels:

| Score | Level    |
|-------|----------|
| ≥ 0.9 | CRITICAL |
| ≥ 0.7 | HIGH     |
| ≥ 0.4 | MEDIUM   |
| < 0.4 | LOW      |

### Step 5: TestCoverageService — Which components need integration tests?

This is Phase 1 of a **two-phase coverage assessment**.

**What it does:** Looks at each impacted component and asks: “Is this the kind of component that should have an integration or end-to-end test?” It does NOT check whether a test already exists.

**Why not check for existing tests here?** impact-service has no access to the test repository. Cloning a repo at every PR event would make the service slow and stateful. The test repo check is done in strategy-service (Phase 2).

### Which component types need integration tests?

| ComponentType | Needs integration test?             | Reason                                             |
|---------------|-------------------------------------|----------------------------------------------------|
| CONTROLLER    | ✅ Yes → API/E2E tests               | Exposes HTTP endpoints used by real clients        |
| SERVICE       | ✅ Yes → INTEGRATION tests           | Contains business logic with side effects          |
| REPOSITORY    | ✅ Yes → INTEGRATION tests (real DB) | SQL queries must be tested against a real database |
| CONFIG        | ✅ Yes → SMOKE tests                 | Misconfiguration can bring down                    |

| ComponentType | Needs integration test? | Reason                                            |
|---------------|-------------------------|---------------------------------------------------|
|               |                         | the whole service                                 |
| MODEL         | ✗ No                    | Data structures — covered by unit tests in the PR |
| UTILITY       | ✗ No                    | Pure functions — covered by unit tests            |

**Required test type determination ( `resolveRequiredTestTypes` ):**

| Condition                                                 | Test type added    |
|-----------------------------------------------------------|--------------------|
| Any CONTROLLER in changed components                      | API                |
| Any SERVICE or REPOSITORY in changed components           | INTEGRATION        |
| Diff contains DB keywords (flyway, ALTER TABLE, etc.)     | INTEGRATION        |
| Diff contains security keywords (auth, oauth, jwt, token) | E2E                |
| Nothing matched                                           | API (safe default) |

**Output:** `CoverageReport` with `level=UNKNOWN`, `source=UNKNOWN`, `untestedComponents=[...]`, `requiredTestTypes=[...]`.

## 7. strategy-service — Phases 2–6: Strategy, Generation, Execution

**Port:** 8082 | **Role:** Real coverage check → decision → BDD generation → code generation → stabilisation

This is the most complex service. It owns the only intelligent decision-making in the system (though it is rule-based, not AI/LLM).

## 7.1 RepoContextService — Cloning and Indexing the Test Repository

`RepoContextService` runs on startup and keeps a local clone of the target test repository fresh. It has two roles: 1. **Context provider** — extracts coding conventions for the test code generators 2. **Coverage index builder** — scans which components already have integration/ E2E tests

### Git operations (without credential prompts)

Every git command is run via `ProcessBuilder` with three layers of credential protection:

```
git -c credential.helper= {command}
// environment variables:
GIT_TERMINAL_PROMPT=0 // no terminal prompt
GIT_ASKPASS=echo      // any askpass program returns empty
```

For token-based authentication, the token is embedded in the HTTPS URL:

```
https://github.com/org/repo
→ https://{username}:{token}@github.com/org/repo
```

### Coverage index — the inverted index

After cloning, `buildCoverageIndex(modulePath)` scans all integration/ E2E test files and builds an **inverted index**.

**What is an inverted index?** A normal index maps documents → words (like a table of contents). An inverted index maps words → documents (like a book's index at the back).

Here, the “words” are component names and the “documents” are test files:

```
coverageIndex:
  "PaymentController" → ["PaymentControllerIT.java", "CheckoutE2ETest.java"]
  "OrderService"      → ["OrderServiceIntTest.java"]
  "UserRepository"    → ["UserRepositoryIT.java"]
```

This structure makes the coverage check **O(1)** — instead of scanning every test file for every component, a single map lookup tells you instantly which tests cover a component.

**Why not just search for the class name across all test files at query time?** With 500 test files and 50 changed components, that would be 25,000 string searches on every single PR. Building once at startup and looking up at query time is dramatically faster.

### Integration/E2E test identification heuristics

`isIntegrationOrE2eFile(Path file)` must distinguish integration tests from unit tests. This matters because unit tests (Mockito + no Spring context) do NOT validate database queries, HTTP contracts, or multi-layer flows.

### Detection criteria (any one match makes it integration/E2E):

| Signal                                                                                   | What it means                                          |
|------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>.feature</code> extension                                                          | Gherkin file — always an acceptance/E2E test           |
| Filename ends in <code>IT</code> or <code>IntegrationTest</code> or <code>IntTest</code> | Naming convention widely used in Java projects         |
| Filename ends in <code>E2ETest</code> or <code>AcceptanceTest</code>                     | Explicit end-to-end naming                             |
| Content has <code>@SpringBootTest</code>                                                 | Boots the full application context — not a unit test   |
| Content has <code>@IntegrationTest</code>                                                | Explicit annotation                                    |
| Content has <code>RestAssured</code>                                                     | HTTP-level testing library — always integration or E2E |
| Content has <code>MockMvc</code>                                                         | Spring MVC test — exercises HTTP layer                 |
| Content has <code>WebTestClient</code>                                                   | Reactive HTTP test — exercises HTTP layer              |
| Content has <code>TestRestTemplate</code>                                                | Spring Boot integration HTTP client                    |

## PascalCase extraction — why use class names?

`extractClassReferences(Path file)` uses the regex `\b([A-Z][a-zA-Z0-9]{2,})\b` to find all PascalCase words in a test file. In Java, all class names are PascalCase. By matching this pattern, the indexer finds every class the test references without needing to parse the full Java syntax tree.

**Why PascalCase specifically?** - Variables in Java are `camelCase` — they start lowercase - Constants are `ALL_CAPS_WITH_UNDERSCORES` — the regex won't match - Class names are `PascalCase` — they start with an uppercase letter

So `\b([A-Z][a-zA-Z0-9]{2,})\b` captures *exactly* the class names (plus annotations without `@`, which is fine because annotations are filtered or used as-is).

**The `EXCLUDED_NAMES` filter (~80 entries) removes:** - Core Java classes: `String`, `List`, `Map`, `Optional`, etc. - JUnit/TestNG annotations: `Test`, `BeforeEach`, `AfterEach`, etc. - Spring annotations: `Autowired`, `SpringBootTest`, `MockBean`, etc. - BDD framework words: `Given`, `When`, `Then`, `Feature`, `Scenario`, etc. - HTTP/testing libraries: `RestAssured`, `MockMvc`, `HttpStatus`, `MediaType`, etc.

What remains after exclusion: application-specific class names like `PaymentController`, `OrderService`, `UserRepository` — exactly what we want.

## Why hard-code the exclusion list rather than using AST parsing?

Full AST parsing (with a library like `JavaParser`) would be 50–100× slower and add a heavy dependency. The exclusion list is fast, deterministic, and covers 95% of cases. The 5% of edge cases (a class named `Given` in the application) are acceptable false positives that do not cause incorrect coverage decisions — they just slightly reduce the precision of the index.

## 7.2 E2ECoverageAnalyzer — Phase 2 Coverage Assessment

E2ECoverageAnalyzer receives the ImpactEnvelope (which carries the UNKNOWN-level CoverageReport from impact-service) and replaces it with a **real** assessment.

```
analyze(envelope):
  index = repoContextService.getCoverageIndex()

  testableComponents = envelope.impactComponents
    .filter(CONTROLLER | SERVICE | REPOSITORY | CONFIG)

  if testableComponents.isEmpty() → GOOD (nothing needs integration tests)

  if index.isEmpty() → UNKNOWN (no repo configured, propagate from impact-service)

  for each component in testableComponents:
    tests = index.get(componentName)
    if tests not empty → covered
    else                → uncovered

  level:
    all covered    → GOOD
    ≥ half covered → PARTIAL
    none covered   → NONE
```

### Coverage level definitions:

| Level   | Meaning                                        | StrategyAgent response            |
|---------|------------------------------------------------|-----------------------------------|
| GOOD    | All testable components have integration tests | Follow other rules (may SKIP)     |
| PARTIAL | Some components covered, some not              | UPDATE_TESTS                      |
| NONE    | No integration tests found for any component   | Force CREATE_TESTS                |
| UNKNOWN | No test repo configured                        | Conservative — follow other rules |



### 7.3 StrategyAgent — The Decision Maker

`StrategyAgent.decide(envelope)` is called once per PR. It first calls `E2ECoverageAnalyzer` to get the real coverage level, then applies a prioritised decision tree.

**Decision tree (evaluated top to bottom, first match wins):**

| Priority | Condition                                                                         | Decision                  | Why                                |
|----------|-----------------------------------------------------------------------------------|---------------------------|------------------------------------|
| 1        | <code>coverage.level == NONE</code> AND <code>untestedComponents</code> not empty | <code>CREATE_TESTS</code> | Repo confirmed: zero tests exist   |
| 2        | All changes are config/dependency AND risk is LOW                                 | <code>SKIP</code>         | Infra changes don't need new tests |
| 3        | PR only changes test files                                                        | <code>SKIP</code>         | Developer already handled it       |
| 4        | Risk is HIGH or CRITICAL                                                          | <code>CREATE_TESTS</code> | Too risky to skip test generation  |
| 5        | <code>NEW_FEATURE</code> detected                                                 | <code>CREATE_TESTS</code> | New features need new tests        |
| 6        | <code>coverage.level == PARTIAL</code>                                            | <code>UPDATE_TESTS</code> | Extend existing tests              |
| 7        | PR includes test file changes                                                     | <code>UPDATE_TESTS</code> | PR touched tests, extend them      |
| 8        | Default                                                                           | <code>CREATE_TESTS</code> | Safe default                       |

**Fallback rules (applied after the base decision):**

| Rule            | Trigger                               | Effect |
|-----------------|---------------------------------------|--------|
| Full regression | <code>confidenceScore &lt; 0.4</code> |        |

| Rule           | Trigger                     | Effect                                        |
|----------------|-----------------------------|-----------------------------------------------|
|                |                             | Sets<br>fullRegressionRequired=true           |
| Expanded scope | risk == HIGH or<br>CRITICAL | Adds transitive dependencies to<br>test areas |

### Confidence score formula:

```
base      = min(1.0, filesChanged / 10.0)
confidence = min(1.0, (base + overallRiskScore) / 2.0)
```

## 7.4 BddGenerator — Creating Human-Readable Test Scenarios

`BddGenerator` takes the `TestStrategy` and produces **BDD (Behaviour-Driven Development) scenarios** in Gherkin syntax.

**What is Gherkin?** Gherkin is a plain-English format for describing test scenarios:

```
Feature: Payment processing
  Scenario: Successful payment
    Given the system is running
    And a valid user session exists
    When the client calls PaymentController
    Then response status is 200
    And response body contains expected data
```

For each `TestRequirement`, `BddGenerator` creates up to 3 scenarios:

| Scenario                             | Type                | When generated                             |
|--------------------------------------|---------------------|--------------------------------------------|
| Successful operation<br>(happy path) | Scenario            | Always                                     |
| Error handling<br>(negative case)    | Scenario            | Always                                     |
| Boundary testing                     | Scenario<br>Outline | Only when <code>API_CHANGE</code> detected |

Tags added automatically: - `@api` / `@ui` / `@mobile` (test type) - `@pr-{prId}` (traceability) - `@auto-generated` (marks it as machine-generated) - `@smoke` (if HIGH or CRITICAL risk)

The BDD PR is a **human review gate** —

`TestPrService.createBddPr()` logs the full Gherkin to the console as a PR payload. Code generation only starts after a human calls `POST /api/strategy/approve-bdd`.

## 7.5 CodegenService — Test Code Generation

`CodegenService` routes each BDD scenario to the appropriate test runner:

| TestType | Runner           | Framework               | Libraries              |
|----------|------------------|-------------------------|------------------------|
| API      | ApiTestRunner    | RestAssured + JUnit 5   | rest-assured , assertj |
| UI       | UITestRunner     | Selenium + ChromeDriver | selenium-java          |
| MOBILE   | MobileTestRunner | Appium + AndroidDriver  | java-client (Appium)   |

Each runner generates a Java source file as a string. The file includes:

1. Repo-specific package name (from `RepoContext`)
2. Common imports found in  $\geq 50\%$  of existing test files
3. `extends {baseTestClass}` if a base class was detected in the repo
4. Agent instruction comments (from `.github/agents/*.md`)
5. `@Test` methods with `GIVEN/WHEN/THEN` comments and assertions from BDD steps

**BDD step → assertion translation (API runner):**

| BDD <code>then</code> step contains | Generated Java assertion                                       |
|-------------------------------------|----------------------------------------------------------------|
| "200"                               | <code>.statusCode().isEqualTo(200)</code>                      |
| "4xx" or "5xx"                      | <code>.statusCode().isGreaterThanOrEqualTo(400)</code>         |
| "2000ms"                            | <code>response.time().isLessThan(2000L)</code>                 |
| anything else                       | <code>// {step} (placeholder comment for human fill-in)</code> |

## 7.6 StabilizationLoop — Run, Fail, Fix

The generated test code is not perfect. It may fail because of timing issues, wrong base URLs, or missing null checks. `StabilizationLoop` automates the repair cycle.

**The loop:**

```

for attempt = 1 to maxRetries (default 3):
  result = testExecutionEngine.execute(script, attempt)
  if PASSED:
    mark script as PASSED
    create final PR
    return
  if attempt < maxRetries:
    apply fix for this attempt
    wait retryDelayMs (default 2000ms)

mark script as ABANDONED
create final PR (for human review)

```

## Three progressive fixes:

**Attempt 1 — Timeout and connection fixes** Applied when error contains "Connection refused", "503", or "timeout". - Adds `RestAssured.config` with `http.connection.timeout=5000ms` and `socket.timeout=10000ms` - Adds `@Timeout(value=30, unit=SECONDS)` to the `@Test` annotation - Injects missing `TimeUnit` and `Timeout` imports

**Attempt 2 — Retry wrapper and null guards** Builds on attempt 1, then: - If error contains "AssertionError": wraps the `// THEN` block in a 3-iteration retry loop with `Thread.sleep(1000)` - Always adds `assertThat(response).isNotNull()` before the status code assertion

**Attempt 3 — Minimal smoke test** Abandons the original test entirely and generates a fresh, minimal class:

```

// AUTO-STABILIZED: Simplified smoke test (original failed 2 attempts)
// Original error: {errorMessage}
public class {OriginalClass}_Stabilized {
  @Test
  void smokeTest() {
    Response response = given().when().get("/api/v1/health").then().extract().response();
    assertThat(response.statusCode()).isBetween(100, 599);
  }
}

```

**The loop always creates a final PR** — even on `ABANDONED`, a  [NEEDS REVIEW] PR is raised so a human can see and fix the partial code.

---

## 8. The Two-Phase Coverage Assessment — In Depth

---

This is the central architectural pattern of the system, worth understanding thoroughly.

### The problem it solves

The question “*does `PaymentController` have integration test coverage?*” requires: 1. Knowing what `PaymentController` is (from the PR diff) 2. Scanning the test repository to find tests that cover it

The PR diff is in `impact-service`. The test repository clone is in `strategy-service`. They are separate services and should not be coupled.

### Phase 1 in impact-service

`TestCoverageService.assess(components, diffs)` runs inside `ImpactEngine` and produces a `CoverageReport` with `level=UNKNOWN`. It identifies *which types of components need integration tests* (by their `ComponentType`) but cannot say whether those tests exist:

```
Input: [PaymentController (CONTROLLER), JwtToken (MODEL), PaymentService (SERVICE)]

Filter: CONTROLLER → needs integration test
       MODEL      → does NOT need integration test (unit test only)
       SERVICE    → needs integration test

Output: CoverageReport(
  level=UNKNOWN,
  untestedComponents=["PaymentController", "PaymentService"],
  requiredTestTypes=["API", "INTEGRATION", "E2E"]
)
```

This is published in the `ImpactEnvelope` to `ImpactResultsQueue`.

### Phase 2 in strategy-service

`E2ECoverageAnalyzer.analyze(envelope)` runs inside `StrategyAgent.decide()` and replaces the UNKNOWN report with a real assessment using the inverted coverage index:

```

coverageIndex contains:
  "PaymentService" → ["PaymentServiceIT.java"]
  (no entry for "PaymentController")

untestedComponents from Phase 1: ["PaymentController", "PaymentService"]

For "PaymentController": coverageIndex.get("PaymentController") = empty → UNCOVERED
For "PaymentService":   coverageIndex.get("PaymentService")   = [...] → COVERED

covered   = ["PaymentService"]    (1 component)
uncovered = ["PaymentController"] (1 component)

covered >= uncovered → PARTIAL

Output: CoverageReport(
  source=REPO_SCAN,
  level=PARTIAL,
  coverageRatio=0.5,
  testedComponents=["PaymentService"],
  untestedComponents=["PaymentController"],
  existingTestFiles=["PaymentServiceIT.java"]
)

```

StrategyAgent then uses this real coverage level: - PARTIAL → UPDATE\_TESTS (PaymentService tests exist; create coverage for PaymentController)

---

## 9. Kafka Manual Acknowledgement — Why It Matters

---

impact-service and strategy-service consume Kafka messages with `AckMode.MANUAL_IMMEDIATE`.

**What this means:** After processing a message, the consumer explicitly calls `ack.acknowledge()` to tell Kafka “I have finished processing this message, advance my offset.”

**Why not use auto-commit?** With auto-commit, Kafka automatically advances the offset after a configurable interval. If the service crashes *after* the offset was advanced but *before* processing completed, the message is lost — no retry.

With manual ack, if the service crashes before calling `ack.acknowledge()`, Kafka re-delivers the message on restart.

The consequence that required custom `KafkaConfig.java` : Spring Boot auto-configuration does **not** create a `kafkaListenerContainerFactory` bean when manual acknowledgement is configured. Without this bean, `@KafkaListener` cannot start — resulting in the error:

A component required a bean named 'kafkaListenerContainerFactory' that could not be found.

Solution: explicit `KafkaConfig.java` in both impact-service and strategy-service:

```
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String> kafkaListenerContainerFactory() {
    ConcurrentKafkaListenerContainerFactory<String, String> factory =
        new ConcurrentKafkaListenerContainerFactory<>();
    factory.setConsumerFactory(consumerFactory());
    factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.MANUAL_IMMEDIATE);
    factory.setConcurrency(3); // 3 parallel consumer threads
    return factory;
}
```

Why `concurrency=3` ? Three parallel threads allow the service to process up to 3 PR envelopes simultaneously. With a single thread, slow PRs (large diffs, many components) would delay all subsequent PRs in the queue.

---

## 10. Multi-Module Maven Build

---

The project uses Apache Maven with a **parent POM** at the root and four child modules.

|                      |                                                                  |
|----------------------|------------------------------------------------------------------|
| QA-ISystem/          | ← parent pom.xml (groupId: nz.co.eroad, version: 0.0.1-SNAPSHOT) |
| ├─ common/           | ← shared models JAR                                              |
| ├─ pr-service/       | ← depends on common                                              |
| ├─ impact-service/   | ← depends on common                                              |
| └─ strategy-service/ | ← depends on common                                              |

**Critical build order:** `common` must be compiled and installed to the local Maven cache ( `~/.m2/repository` ) BEFORE the dependent services compile. The parent POM's `<modules>` section defines this order.

**Build command:**

```
cd QA-ISystem
mvn clean install -pl common && mvn clean package -pl pr-service,impact-
service,strategy-service -am
```

Or simpler (builds all modules in dependency order):

```
mvn clean install
```

**Package namespace:** `nz.co.eroad.qaisystem` — all classes in all modules use this as their root package. This aligns with the company's domain-based package naming convention.

---

## 11. Docker Compose Setup

---

```
services:
  qa-zookeeper: # Kafka's metadata coordinator (required by Kafka)
  qa-kafka:     # The message broker (Kafka)
  pr-service:   # port 8080
  impact-service: # port 8081
  strategy-service: # port 8082
```

All three services depend on `qa-kafka` . Kafka depends on `qa-zookeeper` . Environment variables at the Docker Compose level configure Kafka bootstrap addresses so each service can connect to the message broker inside the Docker network.

---

## 12. Testing the System Locally

---

### Quick path (no repo configured)

```
# 1. Start Kafka
docker-compose up -d qa-zookeeper qa-kafka
```



```
# 2. Start all services (in separate terminals)
cd impact-service && mvn spring-boot:run
cd pr-service && mvn spring-boot:run
cd strategy-service && mvn spring-boot:run

# 3. Send a demo PR
curl -X POST http://localhost:8080/api/pr/demo

# 4. Watch logs in each service terminal
```

## With a real test repo

```
# Set environment variables before starting strategy-service
export TARGET_REPO_TOKEN=ghp_your_token_here
export TARGET_REPO_USERNAME=your_github_username

# Edit strategy-service/src/main/resources/application.yaml:
# aiqa:
#   target-repo:
#     url: "https://github.com/your-org/your-test-repo"
#     branch: main
#     modules:
#       api: tests/api
#       ui: tests/ui
#       mobile: tests/mobile
```

## Approving BDD scenarios (triggering code generation)

After the strategy-service processes a PR, it logs BDD scenarios to the console. Copy the JSON from the log and call:

```
curl -X POST http://localhost:8082/api/strategy/approve-bdd \
  -H "Content-Type: application/json" \
  -d '{...BDD JSON from log...}'
```

## 13. Summary of Heuristics and Why They Were Chosen

| Heuristic                              | Where                                     | Why this approach                                |
|----------------------------------------|-------------------------------------------|--------------------------------------------------|
| PascalCase<br>regex for class<br>names | RepoContextService.extractClassReferences | Java class<br>names are<br>always<br>PascalCase. |

| Heuristic                                                              | Where                                                  | Why this approach                                                                                                                                    |
|------------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                        |                                                        | Faster than full AST parsing; 95%+ precision for enterprise Java                                                                                     |
| Inverted index (component → test files)                                | <code>RepoContextService.buildCoverageIndex</code>     | O(1) lookup per component at query time vs O(nxm) search at every PR                                                                                 |
| Integration test detection via filename patterns first, content second | <code>RepoContextService.isIntegrationOrE2eFile</code> | Filename conventions are faster. Content scan only for projects that don't follow naming conventions                                                 |
| ComponentType from file path keywords                                  | <code>DependencyGraph.detectComponentType</code>       | Java project structure is highly conventional (controllers go in <code>controller/</code> , etc.). Works for 90%+ of enterprise Spring Boot projects |
| Change type detection via regex on changed lines only                  | <code>ChangeTypeDetector</code>                        | Only changed lines are semantically relevant. Scanning unchanged context lines would produce false positives                                         |
| Risk score capped at 0.9                                               | <code>RiskScorer.calculateCoverageScore</code>         | Declaring maximum risk                                                                                                                               |

| Heuristic                                            | Where               | Why this approach                                                                                                                                    |
|------------------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| for coverage factor                                  |                     | without confirmed test absence is overconfident. Repo scan (Phase 2) provides the real signal                                                        |
| Coverage level PARTIAL when covered $\geq$ uncovered | E2ECoverageAnalyzer | Asymmetric threshold is intentional: if more than half is covered, updating is cheaper than creating from scratch                                    |
| Stabilisation: attempt 3 simplifies to smoke test    | StabilizationLoop   | Generated code may be too brittle for the test environment. A smoke test that just checks reachability is better than abandoning silently with no PR |
| 60/80/95% pass probability simulation                | TestExecutionEngine | Represents realistic first-run flakiness. Real implementation would compile and execute the Java code via JUnit Platform Launcher                    |
| acks=all on Kafka producer                           | All producers       | Ensures the Kafka leader AND all                                                                                                                     |

| Heuristic | Where | Why this approach                                                                                |
|-----------|-------|--------------------------------------------------------------------------------------------------|
|           |       | replicas have received the message before acknowledging. Prevents message loss on broker failure |

## 14. Class Reference Table

### common module

| Class                                         | Package        | Role                                                |
|-----------------------------------------------|----------------|-----------------------------------------------------|
| <code>PullRequest</code>                      | model          | Input: PR details from developer                    |
| <code>GitDiff</code>                          | model          | One changed file with line-level detail             |
| <code>ImpactEnvelope</code>                   | model          | Output of impact-service; input of strategy-service |
| <code>ImpactEnvelope.ImpactedComponent</code> | model (nested) | One changed class with type, score, callers         |
| <code>ImpactEnvelope.ChangeType</code>        | model (enum)   | API_CHANGE, BUG_FIX, SECURITY_FIX, ...              |
| <code>ImpactEnvelope.RiskLevel</code>         | model (enum)   | LOW, MEDIUM, HIGH, CRITICAL                         |
| <code>CoverageReport</code>                   | model          | Two-phase coverage assessment                       |
| <code>CoverageReport.CoverageLevel</code>     | model (enum)   | GOOD, PARTIAL, NONE, UNKNOWN                        |
| <code>CoverageReport.CoverageSource</code>    | model (enum)   |                                                     |

| Class        | Package | Role                                             |
|--------------|---------|--------------------------------------------------|
|              |         | REPO_SCAN,<br>UNKNOWN                            |
| TestStrategy | model   | Decision +<br>requirements from<br>StrategyAgent |
| BddScenario  | model   | Gherkin scenarios<br>for BDD review PR           |
| TestScript   | model   | Generated test<br>code + metadata                |
| TestResult   | model   | Execution result<br>from<br>StabilizationLoop    |

## pr-service

| Class                  | Package    | Role                                        |
|------------------------|------------|---------------------------------------------|
| PrServiceApplication   | pr         | Spring Boot entry point                     |
| PRController           | controller | HTTP: /webhook, /<br>submit, /demo, /health |
| PRService              | service    | Enrich + validate +<br>publish              |
| FeatureUpdatesProducer | kafka      | Serialise + send to<br>FeatureUpdatesQueue  |

## impact-service

| Class                    | Package | Role                                               |
|--------------------------|---------|----------------------------------------------------|
| ImpactServiceApplication | impact  | Spring Boot entry<br>point                         |
| KafkaConfig              | config  | Explicit consumer/<br>producer/factory<br>beans    |
| FeatureUpdatesConsumer   | kafka   | Consume<br>PullRequest →<br>trigger ImpactEngine   |
| ImpactResultsProducer    | kafka   | Publish<br>ImpactEnvelope to<br>ImpactResultsQueue |
| ImpactEngine             | engine  | Orchestrate 5-step<br>pipeline                     |

| Class               | Package    | Role                                                    |
|---------------------|------------|---------------------------------------------------------|
| GitDiffParser       | engine     | Raw unified diff → List                                 |
| DependencyGraph     | engine     | Import graph + component typing                         |
| ChangeTypeDetector  | engine     | Regex-based change classification                       |
| RiskScorer          | engine     | Weighted score → RiskLevel                              |
| TestCoverageService | service    | Phase 1 coverage: identify components by type → UNKNOWN |
| ImpactController    | controller | REST: /analyze (sync), /status                          |

## strategy-service

| Class                      | Package  | Role                                           |
|----------------------------|----------|------------------------------------------------|
| StrategyServiceApplication | strategy | Spring Boot entry point                        |
| KafkaConfig                | config   | Explicit consumer/producer/factory beans       |
| TargetRepoProperties       | config   | Typed binding for aiqa.target-repo YAML        |
| ImpactResultsConsumer      | kafka    | Consume ImpactEnvelope → trigger StrategyAgent |
| TestScriptsConsumer        | kafka    | Consume BddScenario → trigger CodegenService   |
| TestScriptsProducer        | kafka    | Publish BddScenario to TestScriptsQueue        |
| StrategyAgent              | agent    | Decision tree + fallback rules                 |
| BddGenerator               | agent    |                                                |

| Class               | Package    | Role                                                         |
|---------------------|------------|--------------------------------------------------------------|
|                     |            | Template-based Gherkin scenario builder                      |
| CodegenService      | execution  | Route BDD scenario to correct runner                         |
| ApiTestRunner       | execution  | Generate RestAssured + JUnit 5 test code                     |
| UITestRunner        | execution  | Generate Selenium test code                                  |
| MobileTestRunner    | execution  | Generate Appium test code                                    |
| TestExecutionEngine | execution  | Simulate (or run) the generated test                         |
| StabilizationLoop   | execution  | Bounded run → fail → fix cycle (max 3 retries)               |
| RepoContext         | execution  | Value object: conventions extracted from test repo           |
| E2ECoverageAnalyzer | service    | Phase 2 coverage: scan repo index → GOOD/PARTIAL/ NONE       |
| RepoContextService  | service    | Clone test repo + build coverage index + extract conventions |
| TestPrService       | service    | Simulate GitHub PR creation (real API call in production)    |
| StrategyController  | controller | REST: /status, / approve-bdd, / refresh-context              |